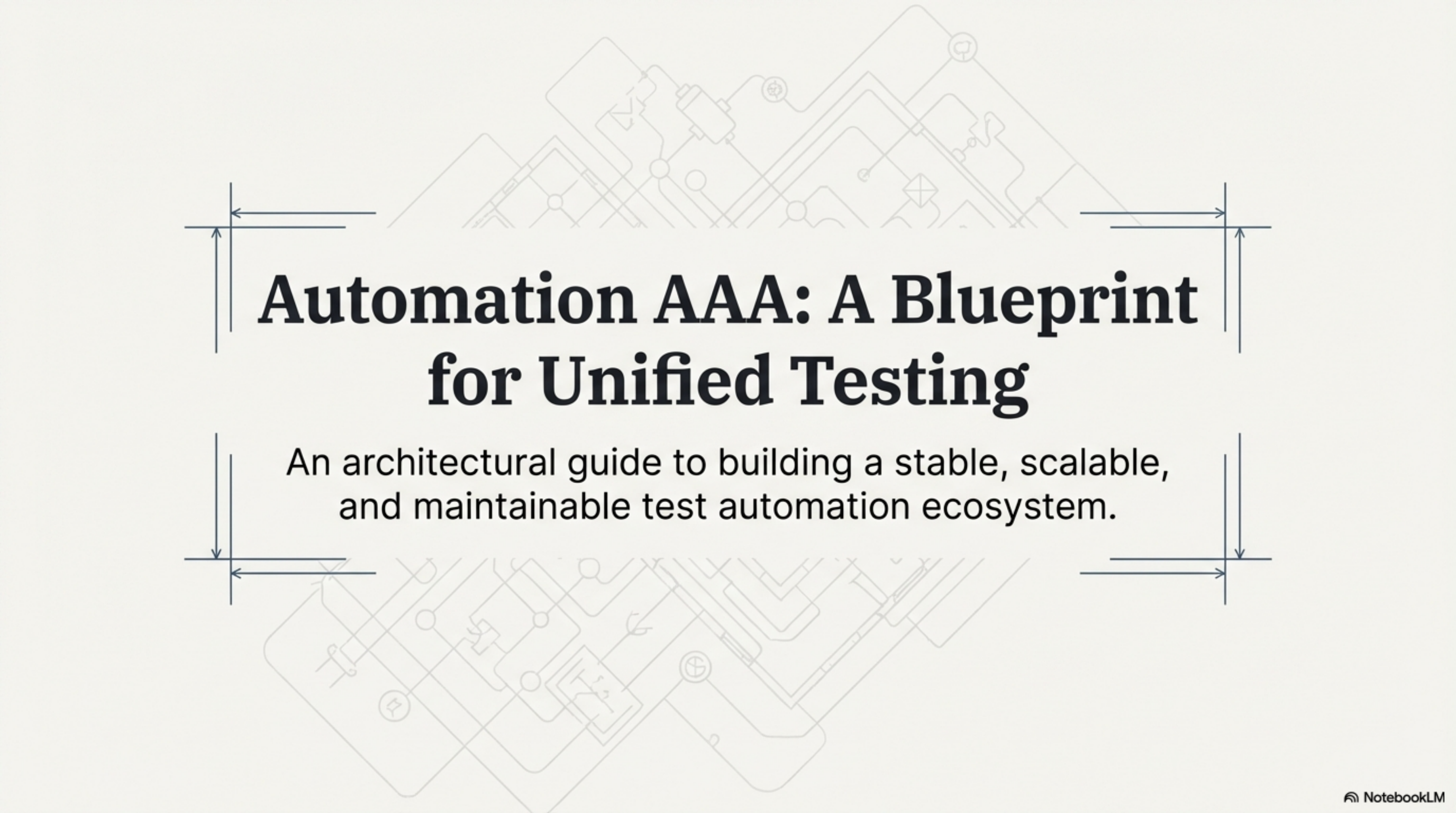




Automation AAA: A Blueprint for Unified Testing

An architectural guide to building a stable, scalable,
and maintainable test automation ecosystem.

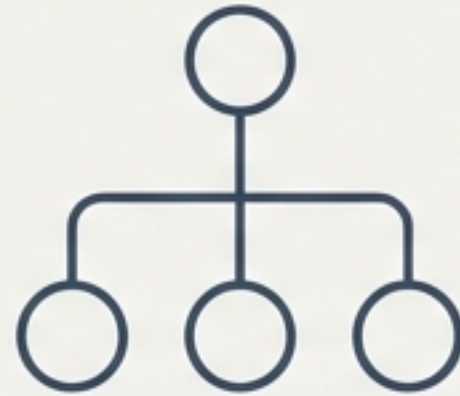
A Cohesive Framework for API, Web, and Desktop Testing

This framework unifies multiple testing domains into a single, cohesive Monorepo. It is built on Nx and Playwright to enforce structure and leverage modern tooling.



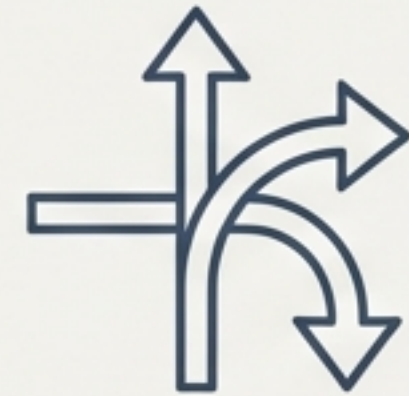
Stability

Achieved through strong typing and the consistent application of the AAA (Arrange-Act-Assert) pattern.



Scalability

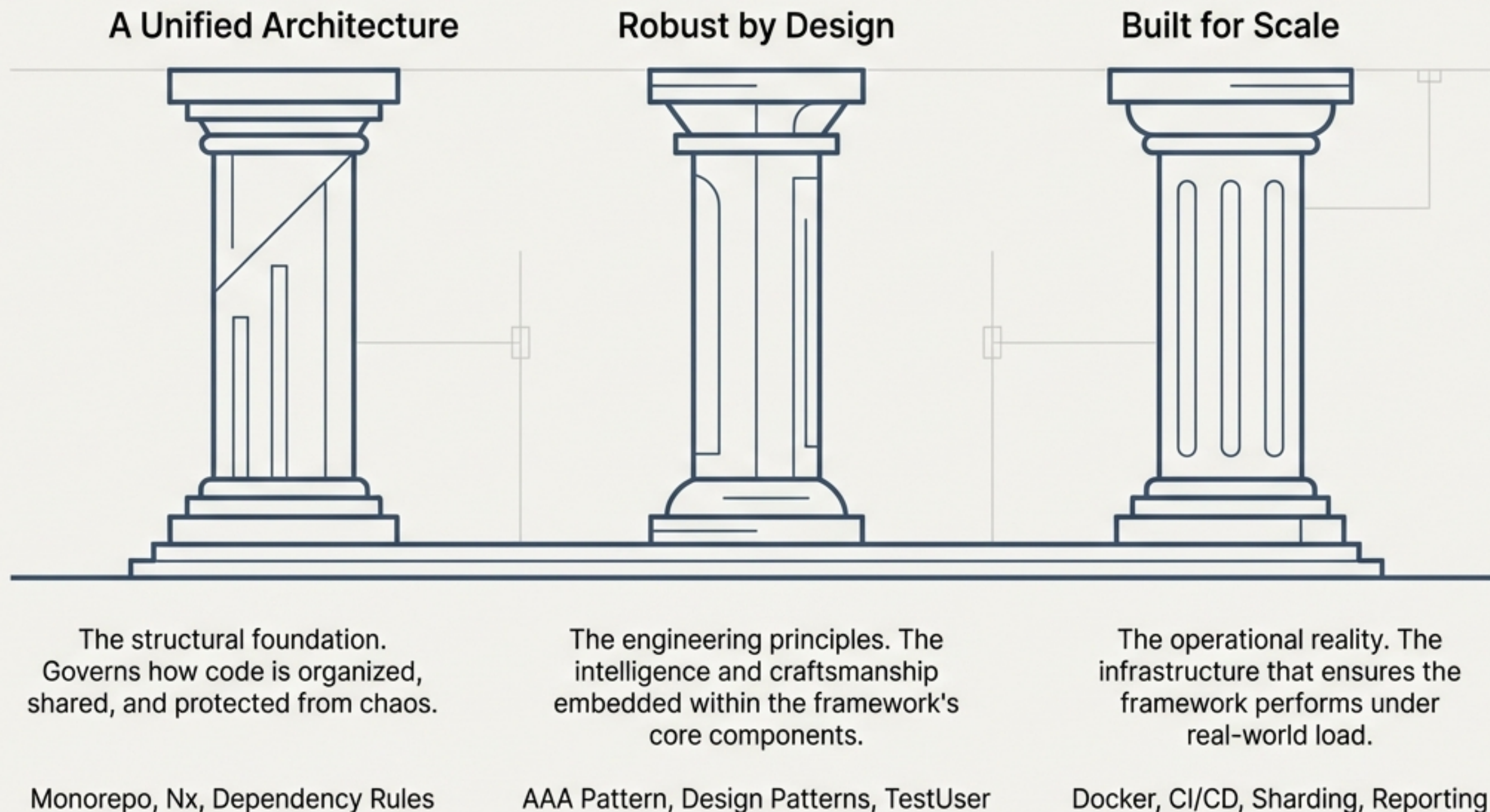
Enabled by a Dockerized architecture that supports sharding for massively parallel execution.



Flexibility

Designed to handle complex scenarios, including native desktop applications and intricate data security tests.

The Three Pillars of the Framework



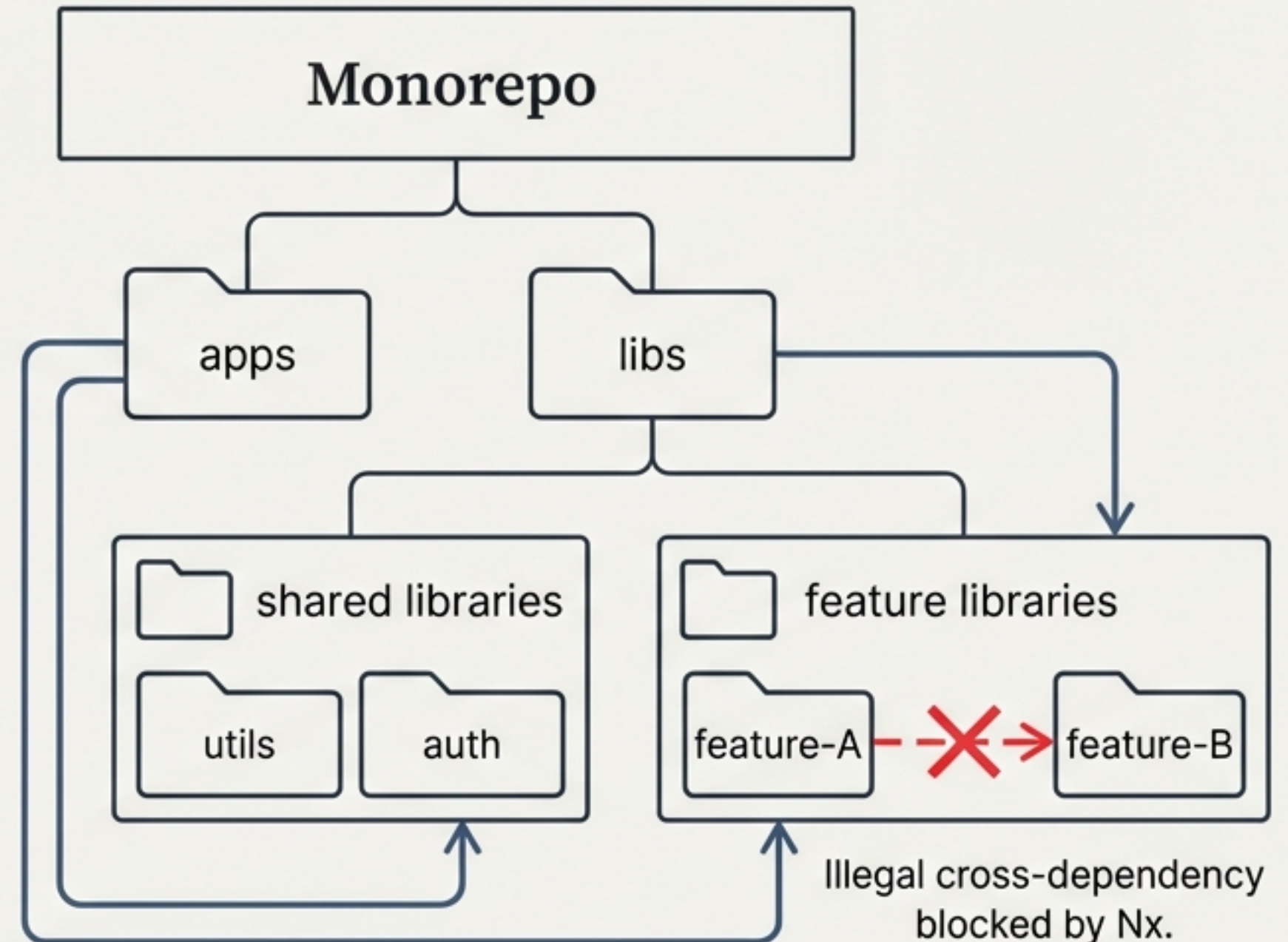
Pillar I: A Unified Architecture Built on an Nx Monorepo

Key Concept

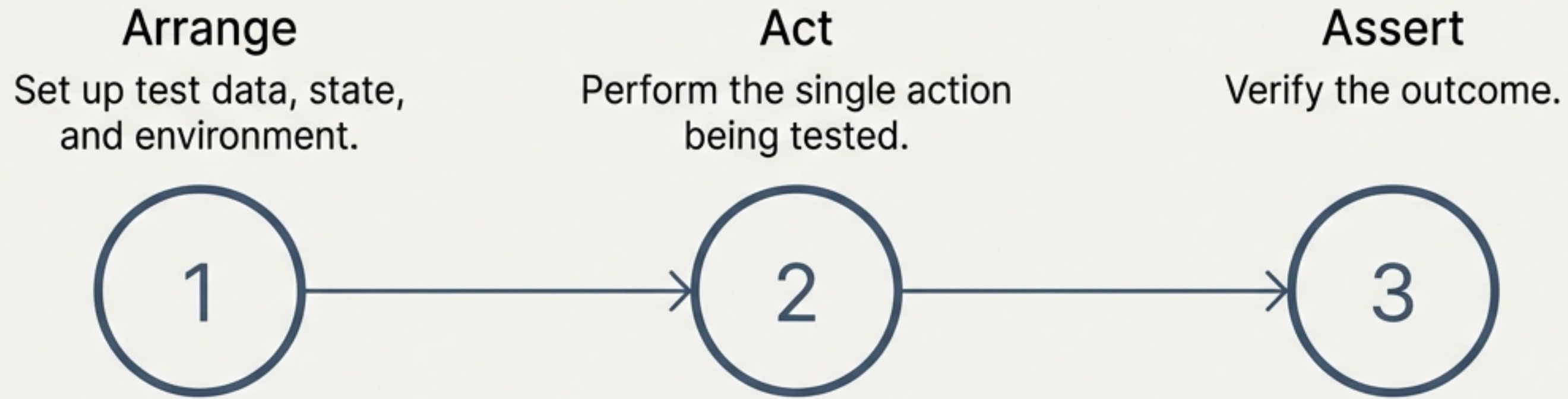
The monorepo is not just a code repository; it's a strategy to eliminate code duplication and enforce architectural consistency across all testing domains (API, Web, Desktop).

Key Feature

Enforced Dependency Rules. Nx's dependency graph prevents tightly-coupled, “spaghetti” code, ensuring libraries remain reusable and the architecture stays clean.



Pillar II: Enforcing Clean Logic with the AAA Pattern



The Framework's Implementation

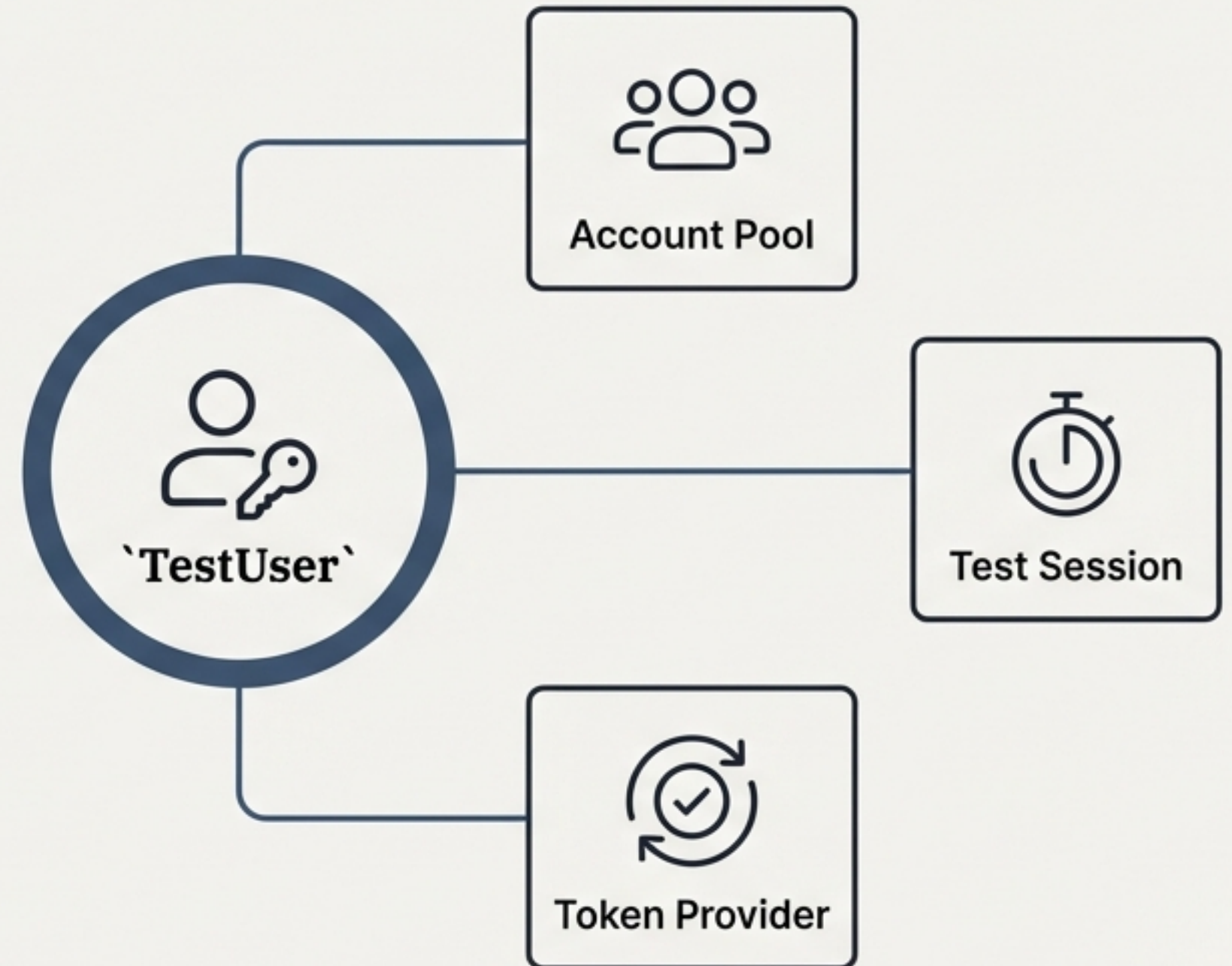
The framework enforces this pattern through a central controller: the ``TestUser``.

The ``TestUser``'s Role

The ``TestUser`` acts as the single entry point for all test interactions. It orchestrates the "Arrange" and "Act" phases, ensuring every test follows a predictable, readable structure. This prevents disorganized test code and improves maintainability.

The `TestUser`: A Central Controller for State and Authentication

- **Lifecycle Management:** Handles the creation, session management, and teardown of a test identity.
- **Account Pooling:** Manages a pool of test accounts to avoid conflicts and enable parallel execution.
- **Authentication Management:** Encapsulates all authentication logic. Features a `TestUserTokenProvider` with automatic token refresh and retry logic with exponential backoff.



Delegating Tasks to Specialized Test Assistants

The `TestUser` doesn't perform actions directly. It delegates tasks to one of four specialized assistants, each with a distinct responsibility.



Design Patterns in Action: The Builder Pattern for APIs

Problem: Constructing complex API request payloads inside a test can be messy, unreadable, and hard to maintain.

Solution: The `InputBuilder` pattern provides a fluent, chainable interface for building request objects.

Without Builder

```
{
  "user": {
    "username": "test-user",
    "details": {
      "role": "admin",
      "status": "active"
    },
    "metadata": { "source": "test-suite" }
  },
  "permissions": [
    { "type": "read", "scope": "all" },
    { "type": "write", "scope": "all" }
  ]
}
```

With `InputBuilder`

```
const apiRequest = new UserInputBuilder()
  .withUsername("test-user")
  .withRole("admin")
  .withCustomPermissions(["read", "write"])
  .build();
```



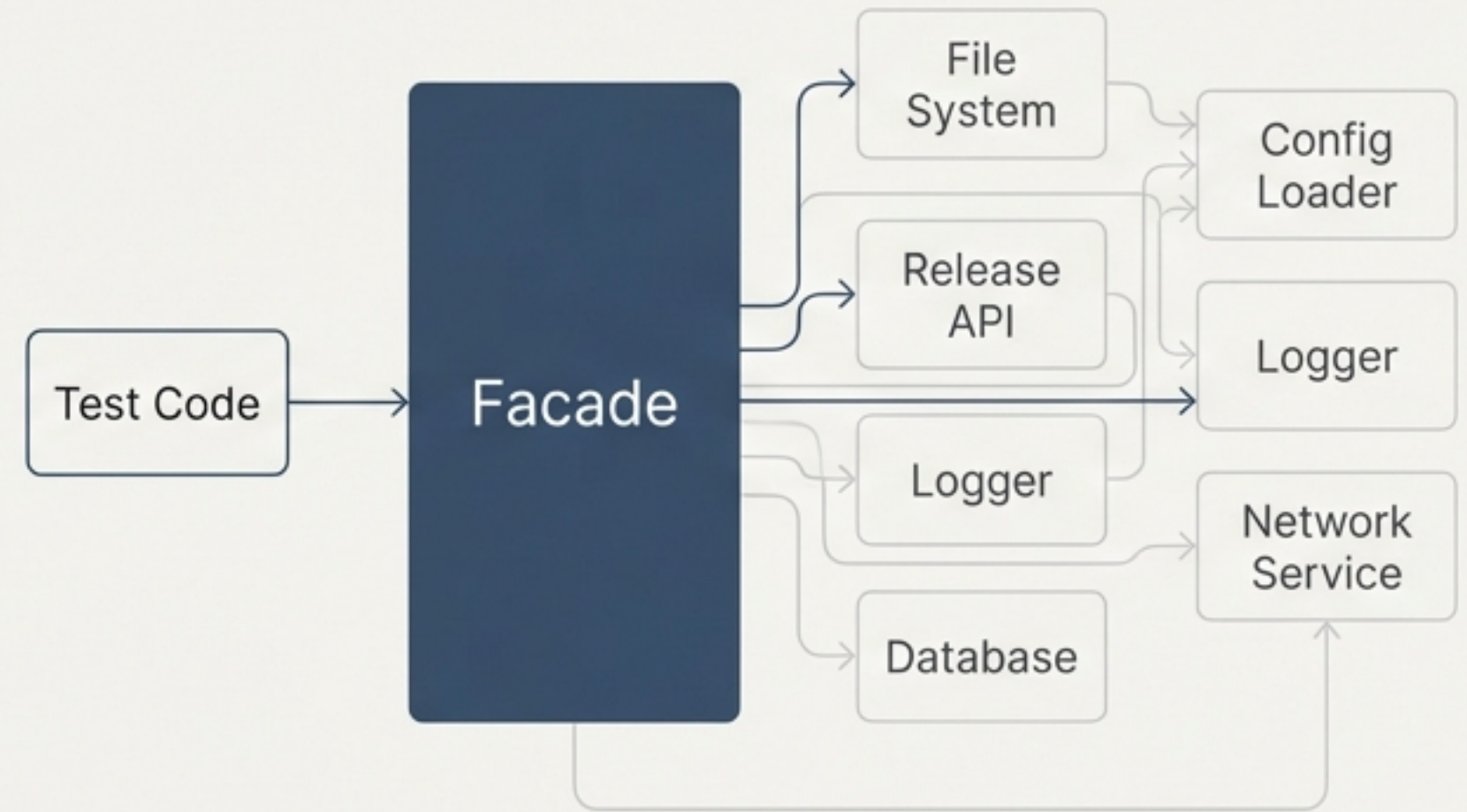
Benefit: Tests become self-documenting, and creating variations of test data is trivial.

Design Patterns in Action: The Facade Pattern for Simplicity

Concept: The Facade pattern provides a simplified, unified interface to a complex set of subsystems. It hides complexity and decouples the test logic from the internal workings of the framework.

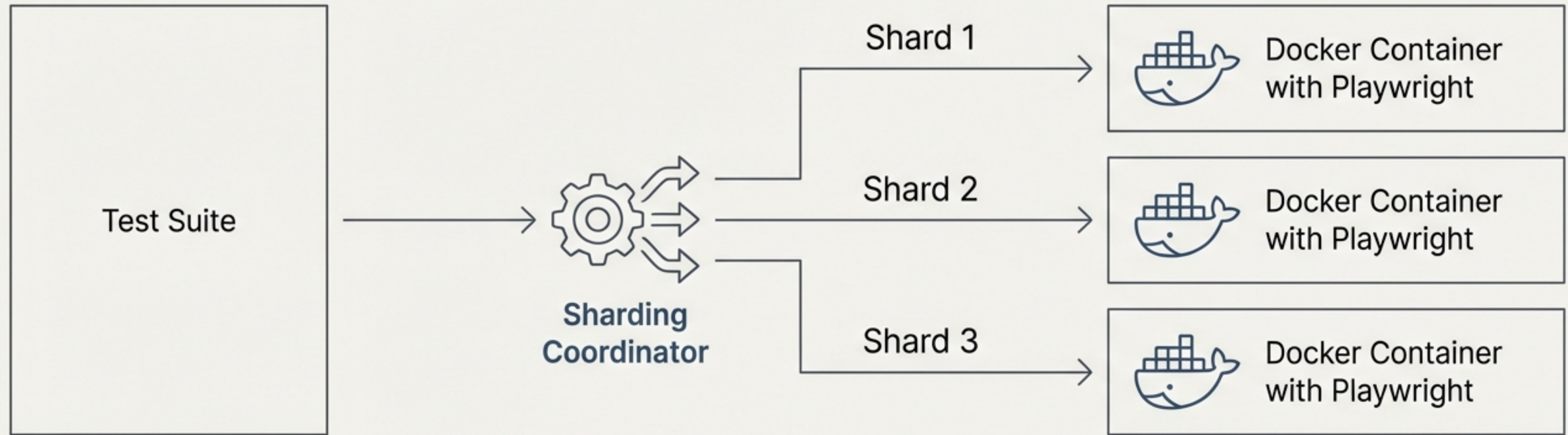
Key Implementations:

1. **`utilManager`**: A central facade that provides easy access to all framework utilities (e.g., file operations, release management) without needing to know their specific locations or implementations.
2. **`Configuration Management`**: A configuration facade abstracts away how environment variables are loaded and merged, providing a simple ``getConfig()`` method to the tests.



Pillar III: Built for Scale with Docker and Sharding

Core Strategy: To accelerate feedback, the framework is designed to run tests in parallel across multiple machines or containers.



- **Dockerized Environments:** Ensures every test run is clean, consistent, and dependency-free. Supports Windows containers.
- **Sharding Strategies:** The framework employs intelligent strategies to coordinate and distribute test files efficiently across parallel workers.

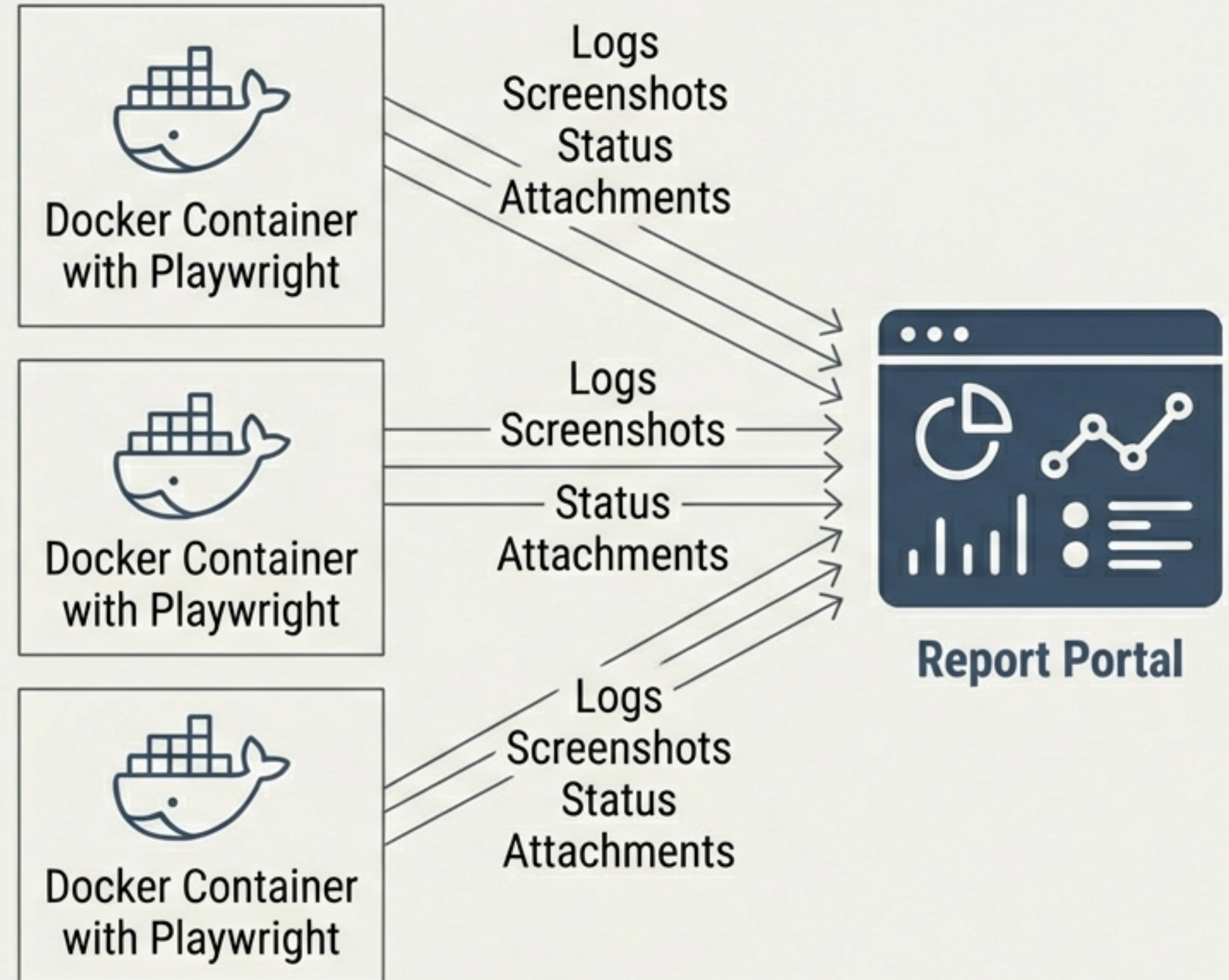
Unifying Results with Centralized Reporting

Challenge: With dozens of parallel test runs, aggregating logs, screenshots, and results is critical.

Solution: Integration with **Report Portal**.

How it Works:

- **'@Step' Decorator:** A custom decorator on functions that automatically logs the step and its parameters.
- **'ExecutionTracker':** A service that manages the context of the test run.
- **Context-Aware Logging:** Logs are automatically tagged with test and step information, making debugging simple.

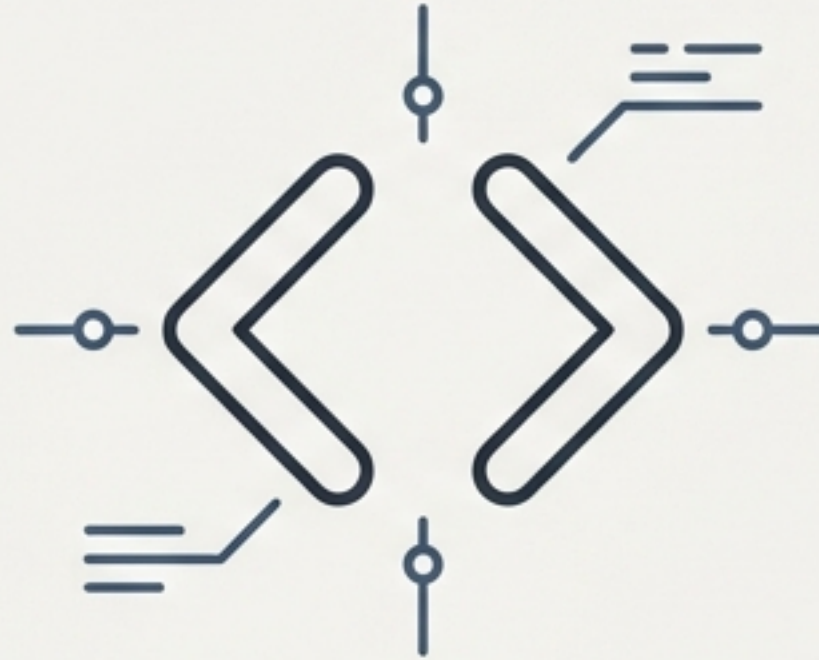


The Framework in Action: Solving Complex Domain Challenges

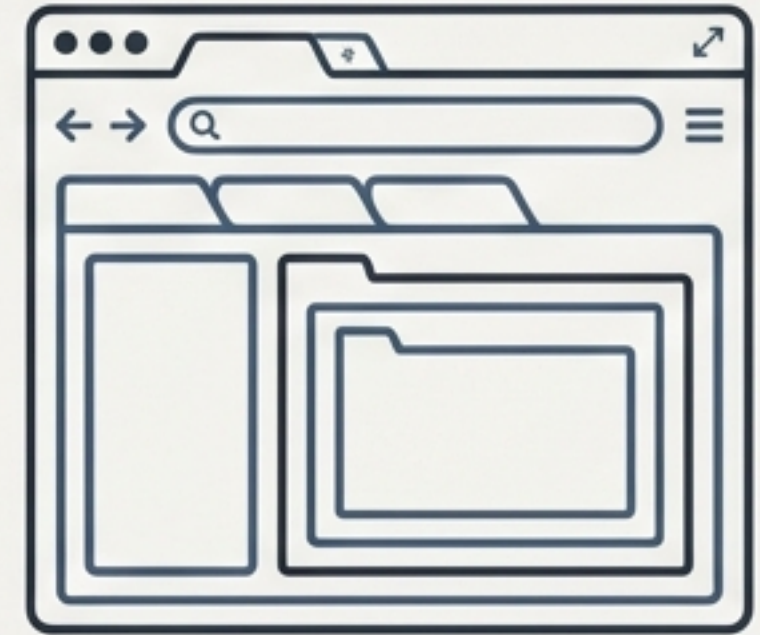
The true test of an architecture is its ability to handle difficult, real-world scenarios. The following examples demonstrate how the framework's core principles are applied to specific testing domains.



Native Desktop Application
Testing (SAB)



Advanced API
Automation



Microsoft 365 and
Complex Web UI

Domain Deep Dive: Desktop Testing with the 'Dual-Driver' Model

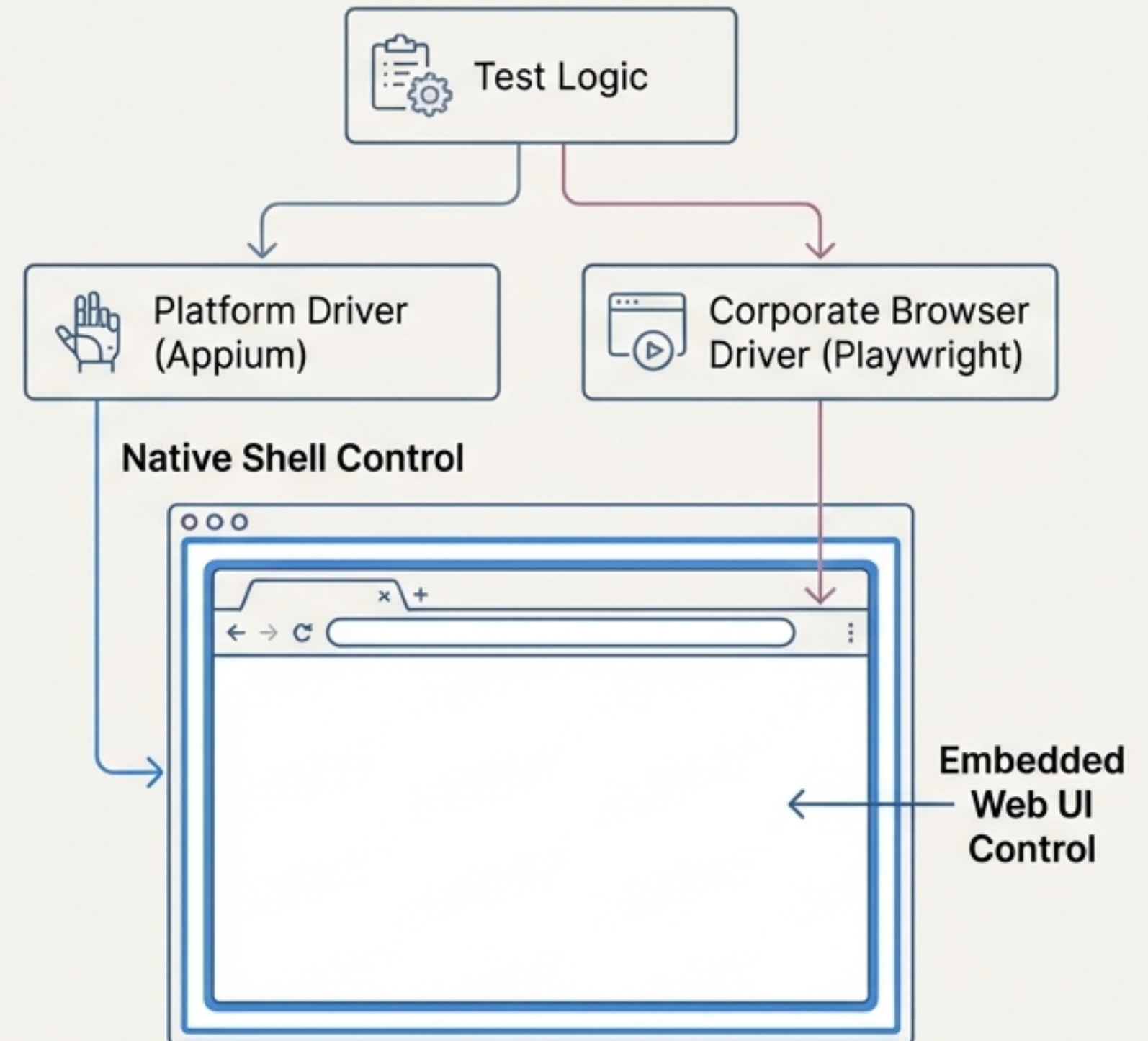
Challenge:

Testing a native desktop application that also interacts with web content requires controlling both the OS and the app's embedded browser context simultaneously.

The 'Dual-Driver' Solution:

The framework orchestrates two drivers working in tandem:

1. **Platform Driver (Appium):** Controls the native application window, clicks buttons, interacts with OS-level dialogues, and manages file systems.
2. **Corporate Browser Driver (Playwright):** Connects to the application's underlying web engine via a Chrome DevTools Protocol (CDP) port to control the web-based UI inside the native shell.



Domain Deep Dive: Conquering Microsoft 365 Automation

The Challenge:

Automating applications like Office Online presents extreme UI complexity.

Framework Strategies for Complex UI:



Canvas Elements: Specialized helpers to interact with non-DOM elements.



Nested iFrames: Robust logic for reliably switching context between multiple frame layers.



Dynamic Context Menus: Strategies for handling menus that appear outside the standard DOM flow.

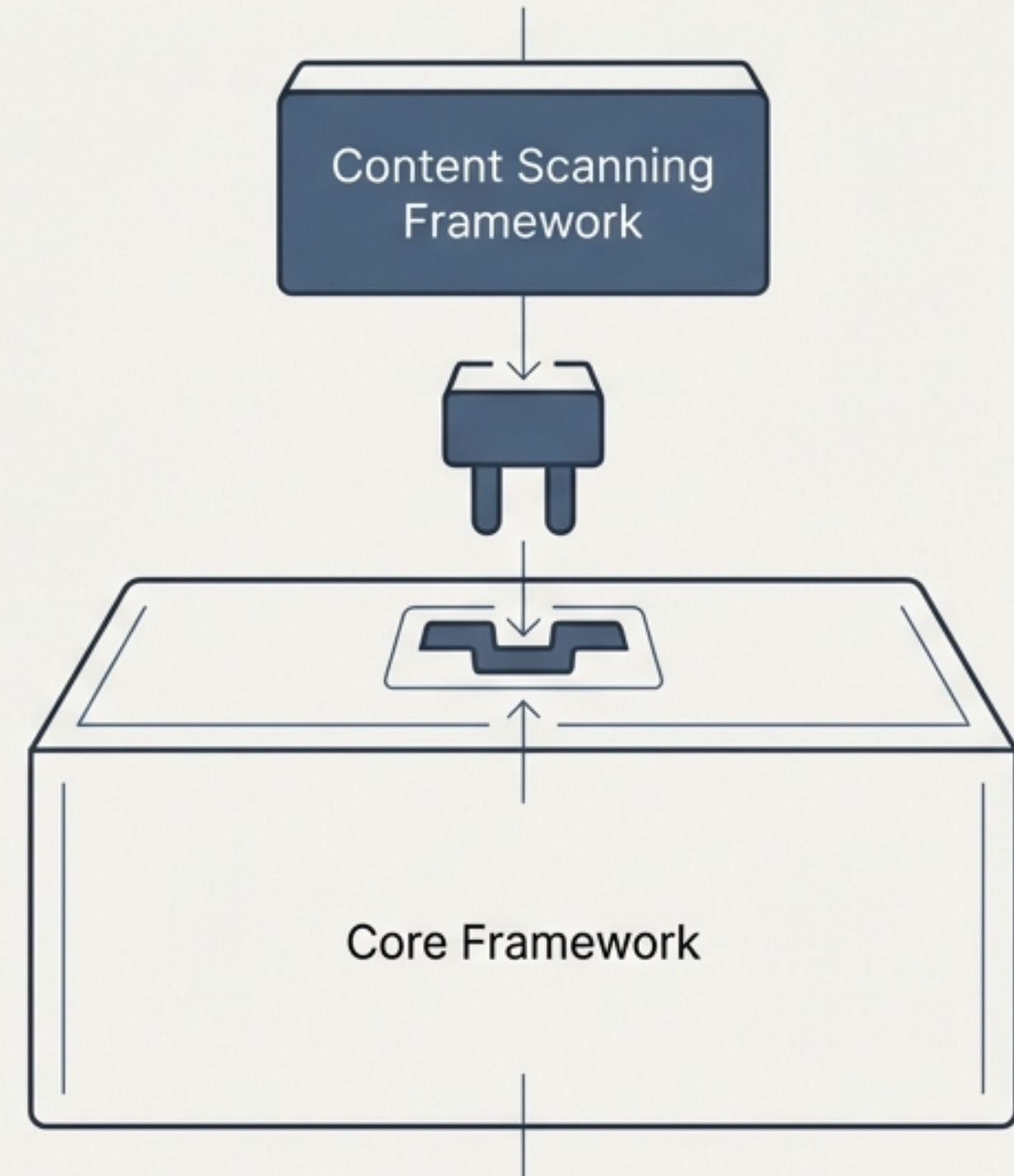
The framework's architecture is flexible enough to handle the most demanding web automation scenarios, proving its resilience.

An Extensible Core for Specialized Testing

- **Concept:** The core framework provides a foundation for building highly specialized, domain-specific testing frameworks.

Example: The Content Scanning Framework

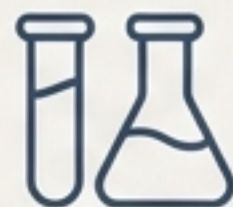
- **Purpose:** Designed specifically for testing Data Loss Prevention (DLP) and Data Security scenarios.
- **Architecture:** Built using a Strategy-based design pattern. This allows new scanning algorithms or data detection methods to be plugged in without changing the core test orchestration.



Managing the Full Lifecycle: DevOps & Runtime

Key Operational Components

- **Global Setup:** A ``SabGlobalSetupManager`` handles environment initialization, Appium server management, and other pre-run requirements.
- **Fixtures:** Provides both isolated and persistent session modes, allowing for clean test states or faster runs where appropriate. Includes helpers for automatic attachment management on failure.



Key Operational Components

- **CI/CD Integration:** Fully integrated into CI/CD pipelines for automated execution.
- **Secrets Management:** Securely handles credentials and sensitive data for different environments.



The Automation AAA Blueprint Summarized

A Unified Architecture



- Single source of truth via Nx Monorepo.
- Maintainability through enforced dependency rules.

Robust by Design



- Clean, readable tests via the AAA pattern and `TestUser`.
- Flexibility and clarity from design patterns (Builder, Facade).

Built for Scale



- High-speed feedback through Dockerized sharding.
- Actionable insights via centralized Report Portal integration.

From Blueprint to Reality: Generating New Components

This repository is a living blueprint. Use the provided tooling to generate new libraries and components that conform to the framework's architectural standards.

To Generate a New Shared Feature Library:

```
npx nx g @nx/js:lib feature-my-logic --directory=libs/shared
```

This ensures that as the framework grows, it grows correctly, maintaining the architectural integrity established from the start.

Beyond Tools: A Philosophy of Maintainable Automation

A successful automation strategy is not about the specific tools used, but about the architectural philosophy that guides their implementation.

The **Automation AAA framework** is a **blueprint** for an ecosystem where **tests are stable**, the **architecture is scalable**, and the **codebase remains maintainable** over the long term. It is a **commitment to engineering excellence in quality assurance**.